Module 11 — Stretch-Tue: Synthetic Load Profiling

> ***Honors Track.*** *This is an Honors Track stretch assignment — not required for program completion, but required for Honors distinction. Stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently.*

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Module 11 Applied Lab. This stretch builds directly on the instrumentation you shipped in the Lab. No challenge tier prerequisite.

What success looks like: By the end of this stretch, you can build an asynchronous `httpx` load generator that ramps concurrent requests across discrete load levels, capture p50 / p95 / p99 latency samples and per-level error counts by reading the instrumented `/metrics` endpoint, identify the latency knee with a quantitative criterion, render a matplotlib chart of latency-vs-load, and write a one-page profile report that makes a data-grounded decision call.

Objective

Author an `httpx`-based load generator that ramps concurrent requests against the M11-instrumented `api`, captures p50/p95/p99 latencies and the error rate at each load level by reading `/metrics`, identifies the **latency knee** (the load level at which p95 starts climbing super-linearly), and writes a one-page profile report with a matplotlib chart of latency-vs-load.

The Lab measured **correctness of instrumentation** — that counters and histograms are wired and that the `/metrics` endpoint is well-formed. This stretch measures **what the instrumented signals tell you about service behavior under load** — the practical reason instrumentation exists.

Setup

Use the standalone template repo:

> ***Stretch-Tue template repo:*** `LevelUp-Applied-AI/m11-s11`. *Click **"Use this template" → "Create a new repository"** on the template page, name your copy `m11-s11`, leave Visibility as Public, and create the repository under your own GitHub account.*

This stretch exercises `httpx.AsyncClient` and `matplotlib`. Both are pinned in the stretch's own `requirements.txt`, so the `pip install -r requirements.txt` step below installs them into your fresh `m11-s11` venv.

```
git clone https://github.com/<your-username>/m11-s11.git
cd m11-s11
git checkout -b stretch-tue-load-profiling
python3.11 -m venv .venv          # use 3.11 — matplotlib wheels do not build on 3
source .venv/bin/activate
pip install -r requirements.txt
```

The stretch ships a small **fixture API** (FastAPI service exposing `/predict` and `/metrics` with an injectable latency floor) so you can run the full pipeline without bringing up the Module 11 Lab stack. Pointing `--base-url` at your Lab's `api` service is supported as an advanced path.

Learning outcomes

By completing this stretch you will be able to:

- Design a concurrent `httpx.AsyncClient` load generator that ramps over discrete load levels and captures per-level latency samples.
- Compute p50 / p95 / p99 latency percentiles from raw samples and read error counts from a `/metrics` Prometheus exposition.
- Define and justify a quantitative criterion for the **latency knee** (e.g., the smallest load level at which p95's per-step derivative exceeds a documented threshold).
- Produce a learner-authored profile report that makes a decision call from data (the knee), not just a chart.

Tasks

1. Implement the load generator

`load_generator.py` defines a callable that takes a base URL, a list of load levels (concurrent-request counts), and a per-level request count, and returns per-level latency samples plus error counts pulled from `/metrics`. Use `httpx.AsyncClient` with `asyncio.gather` to drive concurrency; respect a per-call timeout.

The autograder imports four function names from this module — fill in their bodies, do not rename them. The starter `load_generator.py` ships a `LoadLevelResult` dataclass and `async` signatures the autograder expects; **keep the signatures as they appear in the starter file**, summarized below:

- `async run_one_request(client, base_url) -> tuple[float, bool]` — issue one POST to `{base_url}/predict` and return `(latency_ms, was_error)`. `was_error` is `True` for non-2xx responses or transport exceptions.
- `async run_level(base_url, concurrency, requests_per_level) -> LoadLevelResult` — drive `requests_per_level` requests with at most `concurrency` in-flight (use an `asyncio.Semaphore`), collect latency samples, count errors, and aggregate via `percentile()` into a `LoadLevelResult` (`load_level`, `requests_attempted`, `p50_ms`, `p95_ms`, `p99_ms`, `error_rate`).
- `async run_load_profile(base_url, load_levels, requests_per_level) -> list[LoadLevelResult]` — call `run_level` for each level in `load_levels` and return the ordered list.
- `percentile(samples, q) -> float` — return the `q`-th percentile (`q` ∈ `[0, 100]`) of a samples list. Use a defensible method (`statistics.quantiles(..., n=100, method='inclusive')[q-1]` or `numpy.percentile`) and document your choice in the report.

2. Implement the report writer

`report_writer.py` reads the load-generator's output (`latency_results.json`) and produces:

- `latency-profile-report.md` — a one-page narrative including a per-load-level table with columns `load_level`, `p50`, `p95`, `p99`, `error_rate`.
- `charts/latency-vs-load.png` — a matplotlib chart of p50 / p95 / p99 latency vs load level, referenced from the report.

The autograder imports three function names from this module — fill in their bodies, do not rename them. Match the starter signatures:

- `load_results(path: str) -> list[dict]` — read `latency_results.json` and return its `results` list. Handle a missing or malformed file with a clear error.
- `render_chart(results, chart_path: str) -> None` — plot p50 / p95 / p99 vs `load_level` to `chart_path` (PNG); label axes, include a legend, and annotate the latency knee.
- `render_report(results, chart_relpath: str, report_path: str) -> None` — write the one-page Markdown report at `report_path`. The report **MUST** include a Markdown table with columns `load_level | p50 | p95 | p99 | error_rate` and embed the chart via `![...](chart_relpath)` so the autograder confirms it is referenced.

3. Identify the latency knee

A short section in the report identifies the latency knee with quantitative justification — name the load level, the derivative threshold you used, and the value of p95's step-over-step derivative at that point.

Running the full pipeline

Bring up a target API (fixture is simplest):

```
python fixture_api.py             # listens on http://127.0.0.1:8001
```

Run the load generator:

```
python load_generator.py --base-url http://127.0.0.1:8001 \
    --load-levels 1,5,10,25,50 --requests-per-level 100 \
    --output latency_results.json
```

Build the report:

```
python report_writer.py --input latency_results.json \
    --report-out latency-profile-report.md \
    --chart-out charts/latency-vs-load.png
```

Tests

```
pytest tests/ -v
```

The autograder gates:

- `load_generator.py` defines a callable that returns per-level latency samples plus error counts.
- `report_writer.py` produces `latency-profile-report.md` from a fixture results file.
- The report contains a Markdown table whose columns include `load_level`, `p50`, `p95`, `p99`, `error_rate`.
- The report references a chart image that exists on disk.

Submission

1. Make sure `latency-profile-report.md` and every `charts/*.png` you reference in the report are committed (the autograder checks both the report markdown and the chart files on disk).
2. Commit your changes to `stretch-tue-load-profiling` and push.
3. Open a PR within your fork against `main`.
4. Confirm the autograder workflow succeeds.

Your PR description must include:

- A one-paragraph summary of your load-generator design.
- The load level you identified as the latency knee and the criterion you used.
- Confirmation that `pytest tests/ -v` passes locally.
- Paste your PR URL into TalentLMS → Module 11 → Stretch-Tue 11 to submit this assignment.